

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Pertemuan: Implementasi Sistem Registrasi (Database Integration)

Topik: Register User dengan Hash Password & Validasi

A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Membuat form registrasi.
 2. Mengirim data menggunakan metode POST.
 3. Membuat API Route untuk register.
 4. Melakukan validasi email unik.
 5. Meng-hash password menggunakan bcrypt.
 6. Menyimpan user ke database.
 7. Menampilkan loading dan error handling di frontend.
-

B. Dasar Teori

Alur Sistem Register

User isi form

↓

Frontend kirim POST ke API

↓

API cek method POST

↓

Cek email sudah ada?

↓

Jika tidak → hash password

↓

Simpan ke database

↓

Return response

❏ Mengapa Password Harus Di-Hash?

Password tidak boleh disimpan dalam bentuk plaintext karena:

- Risiko kebocoran data
- Keamanan rendah
- Tidak sesuai best practice

Gunakan:

bcrypt

🔧 C. Langkah Praktikum

♦ Bagian 1 – Membuat Register View

- Buat folder pada **views** dengan nama **register** dan tambahkan 2 file yaitu **index.tsx** dan **register.module.scss**



- Modifikasi file **index.tsx** (pada folder **views/auth/register/index.tsx**)

```
14.praktikum-AuthRegister > my-app > src > views > auth > register > index.tsx > ...
1  import Link from "next/link";
2  import style from "../../auth/register/register.module.scss";
3
4
5  Windsurf: Refactor | Explain | Generate JSDoc | X
6  const TampilanRegister = () => {
7    return (
8      <div className={style.register}>
9        <h1 className={style.register__title}>Halaman Register</h1>
10       <Link href="/auth/login">Ke Halaman Login</Link>
11     </div>
12   );
13 }
14 export default TampilanRegister;
15
```

- Buka file **register.tsx** pada folder auth/register.tsx



- Modifikasi file **register.tsx** (pada folder pages/auth/register.tsx)

14.praktikum-AuthRegister > my-app > src > pages > auth > register.tsx > ...

```

1  import Link from "next/link";
2  import TampilanRegister from "@views/auth/register";
3
4  Windsurf: Refactor | Explain | Generate JSDoc | X
5  const halamanRegister = () => {
6    return (
7      <TampilanRegister />
8    );
9  };
10
11 export default halamanRegister;
12

```

- Modifikasi **register.module.scss**

14.praktikum-AuthRegister > my-app > src > views > auth > register > register.module.scss > .register_title

```

1  .register {
2    padding: 40px;
3  }
4
5  .register_title {
6    font-size: 28px;
7    font-weight: bold;
8  }
9

```

- Tambahkan form inputan pada file **index.tsx** (pada folder views/auth/register/index.tsx) Form berisi:

- Email

```

<div className={style.register__form_item}>
  <label
    htmlFor="email"
    className={style.register__form_item_label}
  >
    Email
  </label>
  <input
    type="email"
    id="email"
    name="email"
    placeholder="Email"
    className={style.register__form_item_input}
  />
</div>

```

- Full Name

```

<div className={style.register__form__item}>
  <label
    htmlFor="Fullname"
    className={style.register__form__item__label}
  >
    Fullname
  </label>
  <input
    type="text"
    id="Fullname"
    name="Fullname"
    placeholder="Fullname"
    className={style.register__form__item__input}
  />
</div>

```

- Password

```

<div className={style.register__form__item}>
  <label
    htmlFor="Password"
    className={style.register__form__item__label}
  >
    Password
  </label>
  <input
    type="password"
    id="Password"
    name="Password"
    placeholder="Password"
    className={style.register__form__item__input}
  />
</div>

```

- Button Register

```

<button type="submit" className={style.register__form__item__button}>
  Register
</button>

```

- Kode keseluruhannya sebagai berikut :

14.praktikum-AuthRegister > my-app > src > views > auth > register > index.tsx > TampilanRegister

```
1 import Link from "next/link";
2 import style from "../../auth/register/register.module.scss";
3
4 Windsurf: Refactor | Explain | Generate JSDoc | X
5 const TampilanRegister = () => {
6   return (
7     <div className={style.register}>
8       <h1 className={style.register__title}>Halaman Register</h1>
9       <div className={style.register__form}>
10         <form action="">
11           <div className={style.register__form_item}>
12             <label
13               htmlFor="email"
14               className={style.register__form_item_label}
15             >
16               Email
17             </label>
18             <input
19               type="email"
20               id="email"
21               name="email"
22               placeholder="Email"
23               className={style.register__form_item_input}
24             />
25           </div>
26           <div className={style.register__form_item}>
27             <label
28               htmlFor="Fullname"
29               className={style.register__form_item_label}
30             >
31               Fullname
32             </label>
33             <input
34               type="text"
35               id="Fullname"
36               name="Fullname"
37               placeholder="Fullname"
38               className={style.register__form_item_input}
39             />
40           </div>
41           <div className={style.register__form_item}>
42             <label
43               htmlFor="Password"
44               className={style.register__form_item_label}
45             >
46               Password
47             </label>
48             <input
49               type="password"
50               id="Password"
51               name="Password"
52               placeholder="Password"
53               className={style.register__form_item_input}
54             />
55           </div>
56           <button type="submit" className={style.register__form_item_button}>
57             Register
58           </button>
59         </form>
60         <br />
61         <p className={style.register__form_item_text}>
62           Sudah punya akun? <Link href="/auth/login">Ke Halaman Login</Link>
63         </p>
64       </div>
65     </div>
66   );
67 };
68
69 export default TampilanRegister;
```

- Modifikasi register.module.scss

14.praktikum-AuthRegister > my-app > src > views > auth > register > register.module.scss > .register > &__form

```
1  .register {
2    min-height: 100vh;
3    display: flex;
4    flex-direction: column; /* ← penting */
5    align-items: center;
6    justify-content: center;
7    background: linear-gradient(135deg, #1e3c72, #2a5298);
8    padding: 20px;
9
10   &__title {
11     width: 100%;
12     text-align: center;
13     font-size: 36px;
14     font-weight: 700;
15     color: #ffffff;
16     margin-bottom: 40px;
17   }
18
19   &__form {
20     background: #ffffff;
21     width: 100%;
22     max-width: 420px;
23     padding: 40px;
24     border-radius: 16px;
25     box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);
26
27     &__item {
28       display: flex;
29       flex-direction: column;
30       margin-bottom: 20px;
31
32       &__label {
33         font-size: 14px;
34         font-weight: 600;
35         margin-bottom: 8px;
36         color: #333;
37       }
38
39       &__input {
40         padding: 12px 14px;
41         border-radius: 10px;
42         border: 1px solid #ddd;
43         font-size: 14px;
44         transition: all 0.3s ease;
45
46         &:focus {
47           outline: none;
48           border-color: #2a5298;
49           box-shadow: 0 0 3px rgba(42, 82, 152, 0.2);
50         }
51       }
52     }
53   }
```

```

54     button {
55         width: 100%;
56         padding: 12px;
57         border-radius: 10px;
58         border: none;
59         background: #2a5298;
60         color: #ffffff;
61         font-weight: 600;
62         cursor: pointer;
63         transition: 0.3s ease;
64     }
65     &:hover {
66         background: #1e3c72;
67         transform: translateY(-2px);
68     }
69 }
70 }
71 }
72
73 /* Animation */
74 @keyframes fadeIn {
75     from {
76         opacity: 0;
77         transform: translateY(15px);
78     }
79     to {
80         opacity: 1;
81         transform: translateY(0);
82     }
83 }
84
85 /* Responsive */
86 @media (max-width: 768px) {
87     .register {
88         &_form {
89             padding: 30px 20px;
90         }
91
92         &_title {
93             font-size: 26px;
94         }
95     }
96 }
97

```

- Jalankan browsernya <http://localhost:3000/auth/register> sehingga tampilan sebagai berikut

Halaman Register

Email

Fullname

Password

Register

Sudah punya akun? [Ke Halaman Login](#)

◆ Bagian 2 – Membuat API Register

- Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```
14.praktikum-AuthRegister > my-app > src > utils > db > TS servicefirebase.ts > ...
 1  import {
 2      getFirestore,
 3      collection,
 4      getDocs,
 5      Firestore,
 6      getDoc,
 7      doc,
 8      query,
 9      addDoc,
10      where,
11  } from "firebase/firestore";
12  import app from "../firebase";
13
14  const db = getFirestore(app);
15
16  Windsurf: Refactor | Explain | Generate JSDoc | X
17  export async function retrieveProducts(collectionName: string) {
18      const snapshot = await getDocs(collection(db, collectionName));
19      const data = snapshot.docs.map((doc) => ({
20          id: doc.id,
21          ...doc.data(),
22      }));
23      return data;
24  }
25
26  Windsurf: Refactor | Explain | Generate JSDoc | X
27  export async function retrieveDataByID(collectionName: string, id: string) {
28      const snapshot = await getDoc(doc(db, collectionName, id));
29      const data = snapshot.data();
30      return data;
31  }
32
33  Windsurf: Refactor | Explain | Generate JSDoc | X
34  export async function signUp(
35      userData: {
36          email: string;
37          fullname: string;
38          password: string;
39      },
40      callback: Function,
41  ) {
42      const q = query(
43          collection(db, "users"),
44          where("email", "==", userData.email),
45      );
46      const querySnapshot = await getDocs(q);
47      const data = querySnapshot.docs.map((doc) => ({
48          id: doc.id,
49          ...doc.data(),
50      }));
51      // console.log("Query result:", data);
52  }
```

```

50     if (data.length > 0) {
51         // user belum ada → boleh daftar
52         // await addDoc(collection(db, "users"), userData);
53         // console.log("User registered:", data);
54         callback({
55             status: "success",
56             message: "User registered successfully",
57         });
58     } else {
59         callback({
60             status: "error",
61             message: "User already exists",
62         });
63     }
64 }
65

```

- Buat file register.ts pada folder api



- Modifikasi file register.ts

```

14.praktikum-AuthRegister > my-app > src > pages > api > TS register.ts > handler
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import { signIn } from '@utils/db/servicefirebase'
3 import type { NextApiRequest, NextApiResponse } from 'next'
4
5 Windsurf: Refactor | Explain
6 type Data = {
7   name: string
8   alamat: string
9 }
10
11 Windsurf: Refactor | Explain | Generate JSDoc | X
12 export default async function handler(
13   req: NextApiRequest,
14   res: NextApiResponse<Data>
15 ) {
16   if (req.method === "POST") {
17     await signIn(req.body, (result: {status: string, message: string}) => {
18       if (result.status === "success") {
19         res.status(200).json({ name: result.message, alamat: "" });
20       } else {
21         res.status(400).json({ name: result.message, alamat: "" });
22       }
23     });
24   } else {
25     res.status(405).json({ name: "Method not allowed", alamat: "" });
26   }
27 }

```

- Modifikasi `index.tsx` pada folder `register` (tambahkan beberapa code)

```
14.praktikum-AuthRegister > my-app > src > views > auth > register > index.tsx > TampilanRegister > handleSubmit
1  import Link from "next/link";
2  import style from "../../auth/register/register.module.scss";
3  import { useState } from "react";
4  import { useRouter } from "next/router";
5
6  Windsurf: Refactor | Explain | Generate JSDoc | X
7  const TampilanRegister = () => {
8      const [isLoading, setIsLoading] = useState(false);
9      const { push } = useRouter();
10     const [error, setError] = useState("");
11     Windsurf: Refactor | Explain | Generate JSDoc | X
12     const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
13         event.preventDefault();
14         const form = event.currentTarget;
15         const formData = new FormData(event.currentTarget);
16         const email = formData.get("email") as string;
17         const fullname = formData.get("Fullname") as string;
18         const password = formData.get("Password") as string;
19         const response = await fetch("/api/register", {
20             method: "POST",
21             headers: {
22                 "Content-Type": "application/json",
23             },
24             body: JSON.stringify({ email, fullname, password }),
25         });
26         // const result = await response.json();
27         // console.log(result);
28         if (response.status === 200) {
29             form.reset();
30             // event.currentTarget.reset();
31             setIsLoading(false);
32             push("/auth/login");
33         } else {
34             setIsLoading(false);
35             setError(
36                 response.status === 400 ? "User already exists" : "An error occurred",
37             );
38         }
39     };
40     return (
41         <div className={style.register}>
42             <h1 className={style.register__title}>Halaman Register</h1>
43             <div className={style.register__form}>
44                 <form onSubmit={handleSubmit}>
45                     <div className={style.register__form__email}>
```

- Buka browser <http://localhost:3000/auth/register> isikan data dan klik register. Jika berhasil maka akan masuk ke menu login

Halaman Register

Email

dino@gmail.com

Fullname

dino

Password

.....

Register

Sudah punya akun? Ke Halaman Login

◆ Bagian 3 – Install bcrypt

- npm install bcrypt --force

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\14.praktikum-AuthRegister\my-app> npm install bcrypt --force
npm warn using --force Recommended protections disabled.

added 3 packages, and audited 447 packages in 5s

146 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- npm install --save-dev @types/bcrypt --force

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\14.praktikum-AuthRegister\my-app> npm install --save-dev @types/bcrypt --force
npm warn using --force Recommended protections disabled.

added 1 package, and audited 448 packages in 3s

146 packages are looking for funding
  run `npm fund` for details

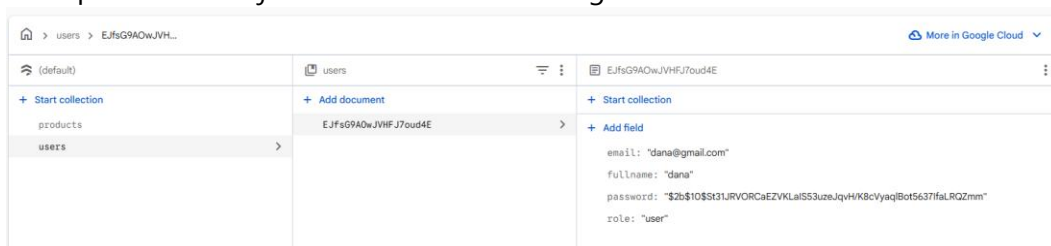
found 0 vulnerabilities
```

- Buka file servicefirebase.ts pada folder src/utils/db dan modifikasi

```
14.praktikum-AuthRegister > my-app > src > utils > db > TS servicefirebase.ts > ...
1  import {
2    getFirestore, collection,
3    getDocs, Firestore, getDoc, doc,
4    query, addDoc, where,
5  } from "firebase/firestore";
6  import app from "firebase";
7  import bcrypt from "bcrypt";
8
9  const db = getFirestore(app);
10
11  Windsurf: Refactor | Explain | Generate JSDoc | X
12  > export async function retrieveProducts(collectionName: string) { ...
13  }
14
15  Windsurf: Refactor | Explain | Generate JSDoc | X
16  > export async function retrieveDataByID(collectionName: string, id: string) { ...
17  }
18
19  Windsurf: Refactor | Explain | Generate JSDoc | X
20  export async function signUp(
21    userData: {
22      email: string;
23      fullname: string;
24      password: string;
25      role?: string;
26    },
27    callback: Function,
28  ) {
29    const q = query( ...
30    );
31    const querySnapshot = await getDocs(q);
32    const data = querySnapshot.docs.map((doc) => ({ ...
33    }));
34    // console.log("Query result:", data);
35
36    if (data.length > 0) {
37      // user belum ada → boleh daftar
38      // await addDoc(collection(db, "users"), userData);
39      // console.log("User registered:", data);
40      callback({
41        status: "success",
42        message: "User registered successfully",
43      });
44    } else {
45      userData.password = await bcrypt.hash(userData.password, 10);
46      userData.role = "user";
47      await addDoc(collection(db, "users"), userData)
48        .then(() => {
49          callback({
50            status: "success",
51            message: "User registered successfully",
52          });
53        })
54        .catch((error) => {
55          callback({
56            status: "error",
57            message: error.message,
58          });
59        });
60    }
61  }
62  }
```

- Jalankan browser <http://localhost:3000/auth/register> dan input data setelah itu klik register

- Buka pada firebase jika berhasil maka data register akan masuk



- Jika user memasukkan data yang sama sistem tidak akan memproses tetapi permasalahannya user memasukkan data yang sama tidak ada pemberitahuan pada layar maka dari itu perlu ada perubahan pada **code index.tsx pada folder views/auth/register**

- Line 41

```

39   return (
40     <div className={style.register}>
41       {error && <p className={style.register__error}>{error}</p>}
42       <h1 className={style.register__title}>Halaman Register</h1>

```

- Line 94 dan 96

```

91     <button
92       type="submit"
93       className={style.register__form__item__button}
94       disabled={isLoading}
95     >
96       {isLoading ? "Loading..." : "Register"}
97     </button>

```

- Line 34 rubah menjadi email

```
14.praktikum-AuthRegister > my-app > src > views > auth > register > index.tsx > TampilanRegister > handleSubmit > response
6   const TampilanRegister = () => {
10  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
26      if (response.status === 200) {
27          form.reset();
28          // event.currentTarget.reset();
29          setIsLoading(false);
30          push("/auth/login");
31      } else {
32          setIsLoading(false);
33          setError(
34              response.status === 400 ? "Email already exists" : "An error occurred",
35          );
36      }
37  };

```

- Modifikasi juga pada register.module.scss

```
14.praktikum-AuthRegister > my-app > src > views > auth > register > register.module.scss > .register > &__error
1   .register {
2       min-height: 100vh;
3       display: flex;
4       flex-direction: column; /* ← penting */
5       align-items: center;
6       justify-content: center;
7       background: linear-gradient(135deg, #1e3c72, #2a5298);
8       padding: 20px;
9
10  &__title { ...
17  }
18
19  &__form { ...
70  }
71
72  &__error {
73      color: red;
74      font-size: 14px;
75      margin-top: 8px;
76  }
77  }

```

- Jika berhasil maka hasilnya seperti berikut

Email already exists

Halaman Register

Email

Fullname

Password

Register

Sudah punya akun? Ke Halaman Login

- Tambahkan loading dengan menambahkan kode pada index.tsx

```
14.praktikum-AuthRegister > my-app > src > views > auth > register > index.tsx > TampilanRegister > handleSubmit
1  import Link from "next/link";
2  import style from "../../auth/register/register.module.scss";
3  import { useState } from "react";
4  import { useRouter } from "next/router";
5
6  Windsurf: Refactor | Explain | Generate JSDoc | X
7  const TampilanRegister = () => {
8    const [isLoading, setIsLoading] = useState(false);
9    const { push } = useRouter();
10   const [error, setError] = useState("");
11   Windsurf: Refactor | Explain | Generate JSDoc | X
12   const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
13     setError("");
14     setIsLoading(true);
15     event.preventDefault();
16   }
17 }
```

- Jika berhasil maka hasilnya akan muncul loading saat klik register

D. Pengujian

Uji 1 – Register Baru

Input:

- Email baru

Hasil:

- Data tersimpan di Firestore
- Password ter-hash
- Redirect ke login

Uji 2 – Email Sudah Ada

Input:

- Email yang sama

Hasil:

- Error 400
- Message: Email already exists

Uji 3 – Method GET

Akses:

/api/register

Hasil:

- 405 Method Not Allowed
-

E. Struktur Database (Firestore)

Collection: users

Field:

Field	Tipe
fullName	string
email	string
password	string (hashed)
role	string
createdAt	timestamp

G. Tugas Praktikum

1. Implementasikan register terhubung database.
 2. Tambahkan validasi:
 - Email wajib
 - Password minimal 6 karakter
 3. Tambahkan role default "member".
 4. Tampilkan pesan error di UI.
 5. Screenshot hasil:
 - Register sukses
 - Email sudah ada
 - Database Firestore
-

H. Pertanyaan Analisis

1. Mengapa password harus di-hash?
2. Apa perbedaan addDoc dan setDoc?
3. Mengapa perlu validasi method POST?

4. Apa risiko jika email tidak dicek unik?
 5. Apa fungsi role pada user?
-

🔲 I. Output yang Diharapkan

Mahasiswa menghasilkan:

- ✓ Form registrasi berfungsi
- ✓ API POST berjalan
- ✓ Email unik tervalidasi
- ✓ Password ter-hash
- ✓ Data tersimpan di Firestore
- ✓ Error tampil di UI
- ✓ Redirect ke login